A Study on the Energy Sustainability of Early Exit Networks for Human Activity Recognition

Emanuele Lattanzi, Chiara Contoli, and Valerio Freschi

***Abstract*—The design of IoT systems supporting deep learning capabilities is mainly based today on data transmission to the cloud back-end. Recently, edge computing solutions, which keep most computing and communication as close as possible to user devices have emerged as possible alternatives to reduce energy consumption, limit latency, and safeguard privacy. Early-exit models have been proposed as a way to combine models with different depths into a single architecture. The aim of this article is to investigate the energy expenditure of a distributed IoT system based on early exit architectures, by taking human activity recognition as a case study. We propose a simulation study based on an analytical model and hardware characterization to estimate the trade-off between the accuracy and energy of early exit-based configurations. Experimental results highlight nontrivial relationships between architectures, computing platforms, and communication link. For instance, we found that early-exit strategies do not ensure energy reductions with respect to a cloud-based solution if the same accuracy levels are kept; nonetheless, by tolerating a 1.5% decrease in accuracy, it is possible to achieve a reduction of around 40% of the total energy consumption.**

***Index Terms*—Energy sustainability, early exit models, human activity recognition, deep learning, distributed intelligence.**

I. Introduction

THE possibility of integrating IoT and embedded computing technologies with deep learning methodologies represents nowadays an appealing direction toward the design of intelligent systems. As a matter of fact, this is witnessed by the wide range of applications where this integration has become key to successful deployments, from health monitoring to assisted living, from smart farming to industrial automation.

However, due to the growing amounts of data that are generated in many application contexts, the batch transmission of all data to the cloud for back-end processing could potentially become critical in terms of energetic efficiency. Moreover, privacy, trustworthiness, and latency issues also prompt the need for solutions alternative or complementary to the cloud-based paradigm [1]. Possible solutions are represented by system architectures aimed at moving computation nearby (or directly on) the devices, thus limiting the continuous transmission of bulks of data to the cloud. The design of IoT systems where some compute and networking services are pushed from the center to the edge of the network, as close as possible to the devices, is also known as *edge computing*; when machine learning capabilities are distributed across the different components of the computing architecture, this is commonly referred to as *edge intelligence* [2], [3].

Indeed, several research challenges have to be tackled to successfully design and deploy edge-intelligent systems. Among these, energetic sustainability and latency certainly represent significant ones. In fact, regarding the contribution in terms of the carbon footprint of these systems, while IoT technologies are widely recognized as key enablers in the path to the reduction of emissions in many contexts, and despite the wide availability of low-power IoT end devices, the overall energy consumption of IoT based systems (encompassing devices and infrastructure), still results into a significant carbon footprint [1], [4].

For what concerns latency issues, these especially arise in real-time systems where tight constraints are imposed on admissible delays that can accumulate during the whole sensing-transmission-processing chain. Applications that make use of deep learning models can be in particular sensitive to this issue, especially in a distributed setting. While deep models are in fact undoubtedly gaining importance with respect to shallow counterparts because of their improved accuracy and feature extraction capabilities, they typically impose a higher computational burden and memory footprint that could in principle prevent their adoption on low-power constrained devices. Resorting to cloud-based inference has been, up to recently, the solution to overcome this issue, which is however not satisfactory either from the latency, energy, and privacy points of view. Indeed, unpredictable delays can be introduced by the different network links (according, for instance, to their congestion), data transmission usually represents a conspicuous fraction of the typical power budget of IoT and mobile devices, while keeping more data close to its source helps mitigate privacy issues.

Early-exit (EE) models, which can be summarized as neural networks architectures with multiple, intermediate exit branches along the in-depth computation path, represent a viable solution that has been proposed in recent scientific literature as an attempt to reduce latency and energy during inference [5]. The main assumption of this approach is that some samples of the dataset can be correctly classified by means of shallow models (i.e., added capacity is not needed for these samples). Therefore, EE methods adaptively decide the depth to be reached by a given sample according to a specific policy; in other words, *easier* samples, that present patterns which permit a correct

Manuscript received 16 December 2022; revised 12 June 2023; accepted 5 August 2023. Date of publication 8 August 2023; date of current version 5 February 2024. The work of Chiara Contoli was supported by European Union - PON Research and Innovation 2014-2020. Recommended for acceptance by Kenli Li. *(Corresponding author: Emanuele Lattanzi.)*

The authors are with the Department of Pure and Applied Sciences, University of Urbino, 61029 Urbino, Italy (e-mail: emanuele.lattanzi@uniurb.it; chiara.contoli@uniurb.it; valerio.freschi@uniurb.it).

Digital Object Identifier 10.1109/TSUSC.2023.3303270

classification in early branches of the network, do not need to complete all the forward processing stages, thus enabling to save computational resources, to speedup inference and to reduce energy consumption.

In this work, we aim at investigating the complex interplay between communication and computation costs in a distributed IoT system capable of performing deep learning inference tasks. In particular, we selected Human Activity Recognition (HAR, for short) as a case study to highlight the performance variations between different system design points (in terms of architectures, communication protocols, and devices). HAR represents a widely studied application for machine learning also by virtue of its diffusion as a building block of many applications, from health to daily activities monitoring tasks. In particular, we focused on activity recognition from inertial measurement units collected from wearable devices, which consists in the recognition of a specific activity (among a set of possible alternatives) performed by users during a given time window. The typical workflow of this machine learning task is based on a cloud-centric computation paradigm: inertial signals (i.e., acceleration and rotational speed) are collected from sensors on board the wearable device (e.g., a smartwatch) and transmitted to a cloud computing platform where powerful servers perform inference, and results are eventually sent back to the user's device. Transmission can be achieved by means of many different communication interfaces (e.g., Bluetooth, WiFi, cable, etc.) which impact the energy and latency system performance. In this work we compare this typical set-up with different distributed configurations that exploit the EE strategy for processing inertial signals with a deep learning model with multiple exit points placed in different devices; each device is connected to the next stage of the forward processing chain through a specific communication channel and is characterized in terms of energy consumption and delay needed to carry out inference. By composing the contribution of all the stages of the EE model we are able to study the system from the point of view of the energy-latency performance and compare it with possible alternative configurations (e.g., different architectures of the distributed system), to evaluate the impact of design choices on the system.

In the above-described context, this work makes the following contributions:

- we derive an analytical formulation of the energy consumption of an early-exit, deep neural network, implemented in a distributed manner when executing a HAR-related inference task;
- we provide a detailed characterization of power and computation capabilities of some representative hardware platforms that can be used as modules to build the HAR computing system, namely: GPU, server, smartphone, singleboard, and tiny wearable device;
- we make use of the analytical model and of the hardware characterization to implement a simulator that estimates the energy consumption of a given EE deep learning model;
- we perform an exploration of the design space by evaluating different configurations, to investigate the energetic sustainability at the system level.

To summarize, this study aims at enhancing the scientific literature related to EE models by analyzing, modeling, and simulating the trade-off between the energy expenditure and the accuracy of a distributed deep learning architecture equipped with early exits.

The remainder of the article is organized as follows: in Section II we report the main contributions in the current scientific literature regarding energy-aware human activity recognition; in Section III we recapitulate some main concepts related to HAR and deep learning with early-exit strategies; in Section IV we describe the mathematical model that can be derived to estimate the energy consumption of a reference sensor based HAR system; in Section V we show how we integrate the formal model with power and computational performance figures of different devices to build a simulator that accurately estimates the energy consumption of the overall system; in Section VI we illustrate the experiments conducted for the empirical investigation and the related results; in Section VII we summarize the main contributions and findings of our work.

II. Related Work

Energy efficiency is a hot topic not only in cloud data centers but also in edge computing. It is gaining momentum because it allows latency reduction and privacy preservation since data are kept closer to where they originate from. In [6] authors surveyed the energy-aware approach in edge computing. The study is carried out considering energy awareness at six different levels, namely: edge hardware design, edge architectures, edge operating systems, edge middleware, applications and services provisioning, and computing off-loading. Energy-aware computation off-loading consists of the study of evaluating whether it is convenient or not to offload the computation from the edge to the cloud, but also the other way around. According to this survey, in this direction, the research community has focused its attention on proposing effective off-loading strategies that have to be designed taking into consideration time constraints, devices, and input tasks. Indeed, off-loading on the one hand reduces the energy consumption locally to end/edge devices, but on the other one energy is consumed to transmit data to the off-loaded destination, and such a process may experience higher latency compared to local processing. Therefore, off-loading must account for a tradeoff between energy consumption and latency experience.

Off-loading in human activity recognition (HAR) is relatively brand-new, and off-loading decision which also takes into account energy consumption is something that is still in its infancy from the exploration point of view. Most of the work in literature performs the off-loading decision mainly based solely on input complexity or classification confidence. Back in 2020, authors in [7] proposed a hierarchical classification process such that a lightweight classifier executes directly on the IoT device, deciding whether to offload the computation to a gateway or to perform it onboard. If computation is carried out onboard, the second level of (a lightweight) classifier is in charge of distinguishing a subset of human activity. Otherwise, if computation is off-loaded to a gateway, a more complex classifier is in charge

of distinguishing the remaining human activity classes. Here, the off-loading decision is based on data features, i.e., is based on the input.

Similar to this hierarchical classification approach authors in [8] proposed a two-stage approach that leverages a decision tree (DT) classifier to recognize easy human activity classes, and a 1-dimensional convolutional neural network (1D CNN) to recognize those activities marked as fallback classes by the former classifier. Despite no off-loading decision to be taken, a form of early exit is implemented, which allows to save energy at inference time. Indeed, the authors also evaluated the energy consumed by their proposed hierarchical model, which is entirely executed on a microcontroller. Another solution that runs entirely on a microcontroller and considers the energy consumed during inference time is the solution proposed in [9]. Here, the authors proposed an early exit neural architecture search (EExNAS) which consists of a search space of 1D CNN backbone architecture, a multi-objective Bayesian Optimization as a search strategy, and classification probabilities as the evaluator to reflect the true confidence values. Their solution is then benchmarked on the wHAR dataset [10] and the energy consumption is evaluated on EFM32, proving that it shows much more resource efficiency and outperforms their baseline in terms of accuracy.

In [11] authors proposed an adaptive human activity recognition architecture that consists of an output block predictor (OBP), implemented as a DT classifier, followed by a multioutput CNN. Such overall multioutput CNN architecture is designed to run on a resource-constrained low-power wearable device. The main goal of this work was to prove that an adaptive architecture is convenient compared to a non-adaptive one. Indeed, the OBP can decide whether to use the first output block (FOB) thereby saving energy consumption and faster inferencing, compared to using the second output, i.e., the backbone architecture. Since the proposed architecture runs entirely on the wearable device, they only need to consider the energy consumed by the device itself to perform the decision via the OBP, the classification performed by the FOB, and, possibly, the classification performed by the backbone.

In [12] authors proposed the Accuracy and Energy-Aware HAR (AE-HAR) model, which considers a light-weight pretrained shallow algorithm executing on a mobile phone, and a heavy-weight algorithm executing on the cloud data center. Each data window acquired by the mobile phone is subject to a decision-making process that considers both the energy and the probabilistic accuracy information to decide whether to process data locally (to the mobile phone) or not. The idea is to minimize the mobile device energy consumption while maximizing the probabilistic accuracy. Our work is similar to [12] in considering energy consumption and probabilistic accuracy in the off-loading decision process.

Compared to the existing literature, instead of considering only the stand-alone wearable device, in this work we consider all the devices involved in the chain for sensor-based HAR system, whose detailed description is provided in Section IV-A. In particular, we used a real early exit HAR network, trained on a publicly available dataset, to emulate the inference execution time and behavior during a real classification task. The early exit HAR network is designed to be spread across the device chain, i.e., from the smartwatch wearable device up to the cloud, crossing opportunistically a smartphone and an access point. The goal of our approach is to characterize the total energy consumption considering the energy consumed by each device along the whole chain, plus the energy consumed when sending/receiving, encrypting/decrypting data, and in the inference process. To reach this goal, a deep characterization in terms of power and computation capabilities of some popular hardware platforms was conducted combining datasheets and the scientific literature; also, a thorough characterization of different communication technologies in terms of power, throughput, and latency was obtained by analyzing the current scientific literature. To the best of our knowledge, this is the first work that attempts to provide such a thorough exploration in the HAR research area.

III. Background

In this section, we briefly introduce the principles and basic concepts of both human activity recognition and early exit of inference.

A. Human Activity Recognition

Human Activity Recognition consists in detecting a set of daily human activities that can range from gestures, such as brushing teeth [13] and hand washing [14], to dynamic and static activities, such as running and standing [15], to cite very few. The reason HAR gained such a momentum stems from the richness of real-world applications that can take advantage of such a research topic. Indeed, healthcare monitoring tailored to personal health, early diagnosis, patient assistance, and rehabilitation can provide valid support to the medical decision process [16], [17]. Other relevant domains are in the field of gaming [18], daily sport/fitness tracking [19], and industrial applications [20], which raised the interest of both industry and academia.

Typically HAR can be achieved via two main approaches: camera-based and (inertial) sensor-based recognition. The first approach consists in applying computer vision techniques to video and/or images. As surveyed in [21], the authors presented a deep analysis of video HAR approaches, classified according to i) the features extraction process, ii) the recognition stages, iii) the source of the input data, and iv) the machine learning supervision level. Instead, the sensor-based approach consists in wearing (usually) small objects, or patches, that can capture people's movements, position, and vital signs, to name a few. As surveyed in [22], sensors for HAR can be grouped into four types: environmental, acceleration, location, and physiological signals. Environmental are those capturing person's surroundings, such as temperature, audio level, and light intensity. Acceleration is a triaxial accelerometer suitable for capturing a person's ambulation activity. Location is the GPS, also suitable for capturing transportation mode. And vital signals are those suitable for capturing heart rate, ECG, and other medical-related parameters.

It is worth mentioning that: i) the video camera can be considered an external sensor; ii) the combined adoption of multiple sensors provides a richer context of information; iii) the sensor's position on people's body (wrist, chest, pocket, etc.) impacts both the acquired data and accuracy with which the activity can be recognized [23], [24]. Moreover, camera-based recognition lift privacy concerns compared to sensor-based recognition. One of the privacy issue deal with people not willing to be recorded and video-monitored. Sensor-based recognition helps in containing privacy issues, without however eliminating the problem. Indeed, secondary information can be gathered by interpreting data collected from the combination of camera and data sensors [25]. For example, authors in [26] proved that information such as age can be inferred by interpreting accelerometers sensors readings.

To achieve activity recognition properly, usually, four steps are carried out. *Step 1* consists of data acquisition, via images/videos or sensors, that are subsequently collected in so-called datasets. *Step 2* consists of data pre-processing, or so-called data segmentation, where data are divided into smaller groups. For data acquired via sensors, this implies splitting time series signals into windows. Instead, for camera-based data, this implies grouping images in smaller groups, eventually for further processing. *Step 3* consists of features extraction, i.e., the information involved in subsequent classification step are selected; selection may yield different results depending on the adopted approach. The *Step 4* consists in the classification process, which is actually achieved via two intermediate steps. The former requires building a classification model, which is in turn subject to a *training* process; the latter consists of the *inference* process, which provides a prediction of the classification. Training and inference can be carried out several times, depending also on the desired level of accuracy.

B. Early Exit of Inference

The recent widespread adoption of Deep Neural Networks (DNNs) in many application domains, such as computer vision, activity recognition, natural language processing, etc., has been motivated by their great ability to learn and extract features showing better performance (e.g., higher accuracy) compared to previous traditional approaches [27], [28], [29]. Such an improvement, however, is paid in terms of higher workloads and memory requirements, which hinter the DNN deployment in resource-constrained environments. To overcome this limitation, three main approaches have been conceived: i) model design optimization, ii) model reduction, and iii) early exit of inference, also known as adaptive inference.

The first approach envisages a network model conceived to be lightweight by design and, possibly, also designed for a specific end device. Examples of such networks are YOLO [30], Squeeznet [31] and Xception [32] to cite a few. The second approach envisages three main methodologies that have been explored in the literature, i.e.: pruning, quantization, and knowledge distillation [33], [34], [35]. Pruning is about removing unnecessary connections between neural network layers; quantization is about reducing the bits involved in weight representation; knowledge distillation is about transferring knowledge from a larger model to a smaller one. The goal of model reduction is threefold: reduce power demand and consumption, reduce memory footprint, and improve computation time. Moreover, those three methodologies might be leveraged in a combined fashion, and are available in several forms of implementations.

The early exit of inference (EEoI) approach was first conceived by Teerapittayanon et al. by proposing BranchyNet [36]. The goal was to overcome the issue of increased latency and energy required by deeper networks during the inference stage. The basic idea behind the EEoI approach is to have a backbone deep neural network equipped with additional side branches beside the backbone branch. Each branch is typically composed of one or more layers followed by an exit point, i.e., a classifier. Since features learned at earlier layers are often enough to provide a correct prediction, there is no need to always go through all the layers of the backbone deep network. Therefore, exiting at intermediate points can significantly reduce computational resource consumption, energy, and inference time, without sacrificing accuracy or other application-specific requirements. The exit criteria adopted by BranchyNet envisages the classification entropy. This implies measuring the confidence of the prediction, that is: if the classifier is confident in the prediction because the entropy is lower than a certain threshold, then it is not necessary to go through further layer processing, and the prediction result is returned at this exit point. Otherwise, the sample continues through the deep network, in the worst case exiting at the last layer of the backbone network.

Despite being a suitable candidate solution to speed-up inference, it entails two critical questions: how do we carefully design and train the backbone network and its relative sub-network exits? How do we choose the exit policy? Those topics are well investigated in [5] where Laskaridis et al. surveyed state-of-the-art techniques to design early exit networks and exit policies. The authors highlighted two early exit design strategies: hand-tuned end-to-end designed networks and vanilla backbone networks. In the first strategy, the model and the network architecture of its early exits are co-designed. In the second strategy, the backbone network design is completely decoupled from those of the early exits sub-networks. The design phase also involves a decision about the number and position of the early exits; these decisions depend on multiple aspects, such as the use case, the exit rate, and the accuracy of each early exit. To place exit points, two approaches are mentioned by the authors: equidistant placement and variable distance across the depth of the network. Choosing the number of exits is difficult because, in case of too many exits points, there is a higher overhead compared to the pure vanilla backbone network case. Instead, if exit points are too sparse, this may cause higher delays before the next output is available.

Laskaridis et al. also mentioned two training strategies: end-to-end training, and intermediate classifiers (IC). In the first approach, the backbone network and the early exit sub-networks are trained jointly, whereas, in the second one, those networks are trained at different stages and independently of each other. First, the backbone is trained and frozen. Subsequently, early exits are added at different points of the backbone and are trained

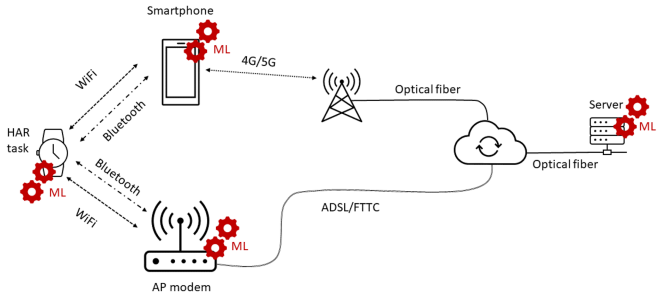

Fig. 1. Reference system for sensor-based HAR.

separately. It is worth mentioning that the IC can be implemented via two different approaches: IC with distillation and IC personalized early exits. The authors also highlighted that the choice of training strategy impacts accuracy and flexibility for target-specific adjustments. The last step of the chain explored by the survey is the deployment on a target device once the network is ready to perform inference on the field. Here, three inference methods are highlighted: subnet-base inference, anytime adaptive inference, and budget adaptive inference. The subnet-base method consists of one single sub-model selection, which is then deployed on the target device. The anytime adaptive inference consists of the deployment of the whole adaptive model, in which each sample early exits based on the confidence of the network prediction and possibly other application-specific requirements. The last method is similar to adaptive inference, but it takes into account also throughput-driven budget.

IV. Modeling the System

In this section, we first present the reference system description and then we provide a formal definition of the activity recognition problem together with the model used to simulate the energy consumption.

A. System Overview

The reference system for sensor-based HAR can be seen in Fig. 1. The HAR task takes input data from a wearable device provided with several sensors such as accelerometers, gyroscopes, and magnetometers and equipped with Bluetooth and WiFi communication interfaces. The smartwatch opportunistically can use an edge hardware, such as: a gateway, a smartphone on which a companion app resides, or directly an infrastructured network offered by an access point of a classic home ADSL/FTTC data link. Both the smartphone and the access point provide the smartwatch the connectivity needed to reach the on-cloud server. The smartphone has been the main device through which to do HAR for years thanks to its pervasiveness and its nature as a personal device but, in the last decade, thanks to the increased computing capacity of microcontroller systems, the smartwatch is taking its place by delegating it to the gateway function. The advantages of using the smartwatch in HAR applications are many: i) the device is practically worn throughout the day; ii) the position with respect to the body is always the same; iii) it is also able to monitor activities that involve only the upper limbs (such as writing, washing hands or teeth, eating, etc.).

In a traditional configuration, which we call *baseline*, the smartwatch collects data from internal sensors and sends it to the remote server for HAR inference through the smartphone or the access point. On the other hand, the early exit of inference (EEoI) approach can be mapped on the reference system by envisioning that early branches can be executed on tiny devices, such as smartwatches, and delegating more complex branches to more powerful units deployed along the connection chain (smartphone and access point), up to the cloud server that can offer potentially unlimited computing power.

B. Formal Model

In general, HAR systems aim at determining the ongoing activities of a person starting from sensory observation data. In particular, starting from a set of sensors $S = \{S_1, S_2, \ldots, S_s\}$, such as accelerometer, gyroscope, etc., which produce a time-series $\{d_{s_t}\}$ of data each, and a set of predefined activities $A = \{A_1, A_2, \ldots, A_m\}$, the HAR system constructs a recognition function F for predicting the activity sequence based on $\{d_{s_t}\}$

$$F(d_{s_t}) = \{A'_1, A'_2, \ldots, A'_t\}, A'_t \in A. \tag{1}$$

Obviously, an instantaneous observation of collected data does not contain sufficient information to recognize the ongoing activity, at this purpose, data is observed for a time window of size w that contains data gathered from time $t1$ to $t1 + w$ seconds. For a given sampling frequency f, the time window contains $n = w \times f$ samples corresponding to n observed values ($d_s = \{d_1, d_2, \ldots, d_n\}$) of the time-series $\{d_{s_t}\}$. Considering a time window W_t as a single record, the HAR task can be represented by a function F^*

$$F^*(W_t) = \{L|L, P_{W_t}\}, L \in A, P_{W_t} \in [0, 1], \tag{2}$$

where L is a label representing the predicted activity and P_{W_t} is its probabilistic accuracy.

For an EEoI HAR system (EEoI-HAR) with a network, composed of a set of exit points $X = \{Ex_1, Ex_2, \ldots, Ex_k\}$, (2) must be rewritten to

$$F^*_e(W_t) = \{L_e|L_e, P_{e_{W_t}}\}, L \in A, P_{e_{W_t}} \in [0, 1]. \tag{3}$$

In literature, several policies have been provided to dynamically drive the exit point selection. In this paper, we make use of the normalized entropy as a measure of the probabilistic accuracy. According to the work presented by Wang et al. in 2017 [37], denoted by $c_{em}(W_t)$ the prediction of the eth exit point on the mth class, and by C the total number of classes, the normalized entropy of the prediction is

$$P_{e_{W_t}} = H[c_{em}(W_t)] \triangleq \frac{1}{\log(C)} \sum_m c_{em}(W_t) \log(c_{em}(W_t)). \tag{4}$$

Assuming that, on average, $P_{1_{W_t}} < P_{2_{W_t}} < \ldots < P_{e_{W_t}}$ $\forall W_t$, we can define a threshold β_e to decide whether to early exit the architecture so that, if at the eth exit point, the $P_{e_{W_t}} > \beta_e$ the inference is concluded and the predicted label can be taken

as actual output; otherwise, data must be forwarded to the next exit point for a more confident prediction. In a HAR system, the sensors S are located in the wearable device, traditionally characterized by low energy availability and low computational power but, moving away to reach the cloud, it is possible to find more powerful and energy-efficient computational resources to install the increasingly complex exit points of the EEoI network. In general, we can consider that the farther we go from the wearable device, the more computationally efficient resources we can find but at a cost of more energy needed to transfer sensor data.

In particular, for an EEoI-HAR system, we can define the effective inference energy $EIE(W_t)$ spent for classifying a single data window W_t as

$$EIE(W_t) = E_{tot_1}(W_t) + \sum_{e=2}^{K} \alpha_e \times E_{tot_e}(W_t), \quad (5)$$

where $E_{tot_e}(W_t)$ is the total energy consumption of the device, providing the eth exit point, for calculating inference on W_t, K is the total number of exit points, and α_e is the conditional probability of exit at point eth given the fact that the actual point has been reached. In particular, at the eth point, α_e strictly depends on the probabilistic accuracy provided by previous $e-1$ exit points, on the current probabilistic accuracy $P_{e_{W_t}}$, and on the user-defined threshold β_e.

More thoroughly, as the probabilistic accuracy represents the probability to match a user-defined threshold $\beta_e = 1$, for a generic exit point eth, we can define

$$\alpha_e = P_{e_{W_t}} \times (1 - P_{1_{W_t}}) \times (1 - P_{2_{W_t}}) \ldots \times (1 - P_{e-1_{W_t}}). \quad (6)$$

This means that the probability of making use of a remote exit point decreases exponentially with the distance from it.

In turn, the total energy contribution of a generic eth exit point $E_{tot_e}(W_t)$ can be represented by

$$E_{tot_e}(W_t) = E_{rx_e} + E_{dec_e} + E_{inf_e} + (1 - \alpha_e) \times E_{tx_e}, \quad (7)$$

which is the sum of the energy consumed by the device for receiving, decrypting data, making the inference, and, in case of not satisfactory classification accuracy, sending data to the next point. Notice that, for the wearable device (e=1), which is in charge of executing the first exit point Ex_1, also sampling (E_{samp}) and encryption (E_{enc}) energies must be accounted, the latter only in case of not satisfactory classification accuracy, while this does not apply to the other devices in the chain In conclusion, combining (5) and (7)

$$EIE(W_t) = E_{samp} + E_{inf_1} + (1-\alpha_1) \times \Big[E_{enc} + E_{tx_1}\Big] + \cdots$$
$$\sum_{e=2}^{K-1} \left\{ \alpha_e \times \Big[E_{rx_e} + E_{dec_e} + E_{inf_e}\Big] + \cdots (1 - \alpha_e) \times E_{tx_e} \right\}$$
$$+ \alpha_k \times \Big[E_{rx_k} + E_{dec_k} + E_{inf_K}\Big]. \quad (8)$$

For completeness, it should also be noted that, in the worst case, the inference process ends at the last exit point (K), which corresponds to the backbone network. Here, the result of the inference is still used even though it would not meet the user-defined threshold β_e.

TABLE I
STATE DIAGRAM'S NOTATION

	Description
Samp.	Sampling
W_t full	Time window full, i.e., 128 collected samples
Inf_1	Inferencing at stage 1 (- smartwatch -)
Enc.	Encryption
T_{X_1}	Transmitting data at stage 1 (from smartwatch to smartphone/AP)
R_{X_2}	Receiving data at stage 2 (- smartphone/AP -)
Inf_2	inference at stage 2
T_{X_2}	Transmitting data at stage 2 (from smartphone/AP to Cloud server (GPU))
R_{X_3}	Receiving data at stage 3 (- Cloud server (GPU) -)

V. SYSTEM CHARACTERIZATION

To estimate the total energy consumption of the reference system described in Fig. 1, starting from the formal model described so far, we derive a power state model which is represented by the diagram of Fig. 2, and whose notation is listed in Table I.

Each state of the diagram is assumed to be annotated with its average power consumption. Transitions from states are triggered either by incoming events, such as the reception of data from the previous node, by a timer as in the case of the sampling state, or by the filling of the window W_t for the inference in the smartwatch. In general, for a given reference system, the functional diagram has to be characterized in terms of average power consumption and time spent in each state but, for a EEoI network, also the probabilistic accuracy of each exit point needs to be characterized. In this work we have used a real EEoI-HAR network, trained on a publicly available dataset, to emulate the inference execution time and behavior during a real classification task while the power and time characterization of the devices has been derived from datasheets and the scientific literature.

A. The EEoI-HAR Network

Fig. 3 shows the architecture of the long-short term memory (LSTM) network used in this work with its three exit points. The network takes in input the six signals gathered from the triaxial accelerometer and gyroscope provided by the smartwatch. These signals are arranged by a sequence input layer in a six-channel data chunk that is forwarded to the subsequent LSTM layers. In particular, the backbone network contains four consecutive LSTM layers each of which is followed by a dropout layer (not shown in the figure). The last part of the network is made up of three different layers of fully connected neurons that act as a multi-layer Perceptron. Also in this case, the three dropout layers which intersperse the neurons layer are not shown. The proposed network then terminates with a standard layer that computes the softmax function used by the classification layer to calculate the cross-entropy loss and to infer the activity label. Starting from the backbone network just described, we defined three exit points with increasing complexity. The first, which has been designed to be executed on the wearable device, just after the first LSTM layer, containing 64 hidden units, is followed by a single fully connected layer (6 hidden neurons) which produces the output for the softmax and the classification layer. The second exit point

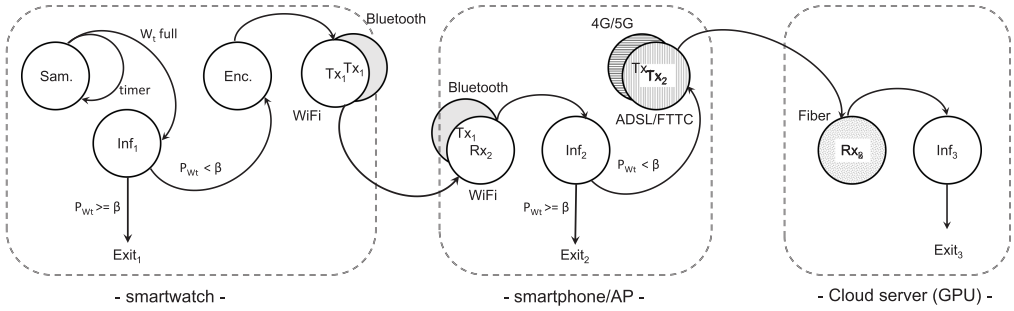

Fig. 2. State diagram of the energy model of the sensor-based HAR system.

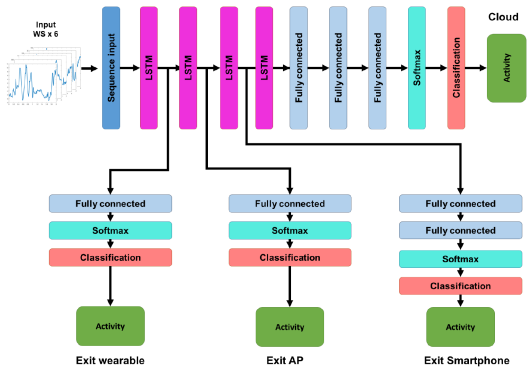

Fig. 3. Architecture of the EEoI LSTM network with its three exit points.

(*Exit AP*) includes the same layer but it takes as input the signals produced by the second LSTM layer and is dedicated to the access point. Finally, the exit point designed for the smartphone originates after the third LSTM layer and, unlike the first two points, it contains an extra fully connected layer provided with 256 hidden neurons.

B. Devices Characterization

A deep characterization in terms of power and computation capabilities of some popular hardware platforms has been conducted. In particular, we divided the entire computer ecosystem into five main categories namely: i) GPU; ii) server; iii) smartphone; iv) singleboard; v) tiny wear. For each category, we collect the computation and power capabilities of several popular top of the range platforms. For what concerns the computational performance, we refer to the values expressed in GFLOPS, obtained by the device in executing Geekbench 4 benchmarks [38]. For some singleboard computers and for tiny wear devices, for which the results of Geekbench 4 are not available, the computation capabilities have been calculated following the general formula:

$$GFLOPS = CPU_{freq} \times NumCores \times instrCycle. \quad (9)$$

Table II reports the collected values of the hardware platforms' performances grouped into the five categories and the power consumption retrieved on its public datasheets. Interestingly, the computation performances cover a range that extends over seven orders of magnitude while the power consumption extends only over four which is proof of the great energy efficiency achieved by the most powerful devices.

TABLE II
SOME POPULAR HARDWARE PLATFORMS AND THEIR COMPUTATIONAL PERFORMANCE

	Device	Performance [GFLOPS]	Power [Watt]
GPU	NVIDIA A100	312000	300
	NVIDIA RTX 4090	8258	450
	NVIDIA RTX 4000	7119	160
	NVIDIA Jetson AGX	5300	50
	NVIDIA Jetson xavier	2800	30
	NVIDIA Jetson nano	500	10
server	Intel Xeon Platinum 8375	5370	300
	Intel Xeon Gold 6230	3920	300
	AMD Ryzen Threadripper	3732	280
	Intel i-9 12900KS	1560	150
	Intel i-7 9800X	1290	165
	AMD Ryzen 9 3950x	1038	105
	AMD Ryzen 7 5800x3D	765	105
	intel i-5 126000KF	755	125
	AMD Ryzen 5	603	45
	Intel i-3 9530KF	397	91
smartphone	Apple A15 Bionic	249	6
	Qualcomm SM8450	165	9
	Samsung Exinos 2100	157	9
	Apple A10X Fusion	141	5
singleboard	Raspberry Pi 3	8	4
	Thinker S	3.6	2.3
	BeagleBone AI	3	7
tiny wear	Espressif ESP32	0.5	0.2
	Mikroe hexiwear	0.12	0.1
	STM32 L5	0.1	0.1

For this purpose, Fig. 4 shows the energy efficiency expressed in GFLOPS/Watt of the devices. As expected, the GPU category reports higher efficiency with the strong supremacy of the NVIDIA A100 which counts more than 1,000 GFLOPS per Watt. At the same time, however, it is possible to note that the devices that equip current smartphones have surprisingly higher efficiency concerning server computers, probably resulting from energy-aware cutting-edge technological solutions. What is striking, however, in categories *singleboard* and *tiny wear* is the high variability of performances which makes one think of appreciably different technological solutions.

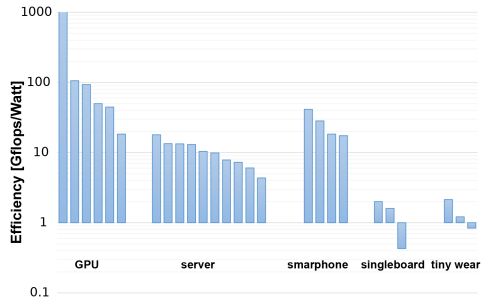

Fig. 4. Energy efficiency of the devices of Table II.

TABLE III
CHARACTERIZATION OF COMMUNICATION AND COMPUTATION CAPABILITY OF THE REFERENCE SYSTEM DEVICES

	power [Watt]	performance [GFLOPS]	Efficiency [GFLOPS/Watt]
computation			
Tiny Wear	0.15	0.24	1.60
Singleboard	4.40	4.87	1.11
Smartphone	7.25	178.00	24.55
Cloud	220.00	56000.00	254.55
communication			
	power [Watt]	**throughput [Mb/s]**	**latency [ms]**
Bluetooth	0.43	1	100
WiFi	0.79	40	10
4G/5G	0.50	150	25
DSL	4.00	20	35
Fiber	5.00	10000	5

From Table II we computed the average performances of each device category, shown in Table III, which have then been used to estimate the reference system energy consumption. In particular, the four categories directly match the device of our reference system where the smartwatch falls into *tiny wear*, the AP in *singleboard*, and so on. Notice that, in order to reduce the degree of freedom of the system, we have not included hardware power models to account for power management strategies or other context-varying conditions, as discussed in references [39] and [40].

Table III also reports a thorough characterization of different communication technologies in terms of power, throughput, and latency obtained by analyzing the current scientific literature [41], [42], [43], [44].

C. Task Characterization

To simulate the energy consumption of the EEoI-HAR system, the classification tasks represented by each exit point must be deeply characterized in terms of the computational workload. Once this has been measured, using the parameters in Table III, it is possible to calculate the average execution time of each task from which, in turn, to obtain the energy cost.

Table IV shows the characterization results of the execution tasks involved in the reference system. For each task, we reported the computation workload expressed in GFLOP obtained by performing each task on a desktop computer with known computational performance and measuring the execution times. For a given load, the execution time on a particular device is then computed using the theoretical computation performance of Table III. For instance, concerning the wearable device, we computed the task parameters of the data encryption and of three different classification models with a decreasing load. Each model is described using the structure of the corresponding EEoI network branch, where xxL means the number of hidden units on a LSTM layer, and yyF is the number of hidden neurons on a fully connected layer. For instance, the model installed on the smartphone, corresponding to the second exit point, is built by a stacked network containing three LSTM layers with, respectively, 64, 128, and 256 internal units, followed by two fully connected layers of 128 and 6 hidden neurons. Concerning the wearable device, starting from a classification model composed of an LSTM layer with 64 hidden units and a fully connected layer with 6 neurons (*Load 1*), we have defined two further models with decreasing complexity by halving each time the number of components inside the LSTM and fully connected layers (*Load 1/2* and *Load 1/4*).

TABLE IV
PARAMETERS CHARACTERIZATION OF EACH EXECUTION TASK INVOLVED IN THE HAR SYSTEM

	Model	Load [GFLOP]	Time [s]	Accuracy
Wear Encry.	-	0.025	1.04E-01	-
Wear Load 1	64L 6F	0.299	1.24E+00	0.897
Wear Load 1/2	32L 6F	0.149	6.22E-01	0.891
Wear Load 1/4	16L 6F	0.075	3.11E-01	0.732
AP	64L 128L 6F	0.687	1.41E-01	0.936
Smartphone	64L 128L 256L 128F 6F	1.709	9.60E-03	0.944
Cloud	64L 128L 256L 512L 512F 512F 6F	4.699	8.39E-05	0.951

Finally, the last column of the table shows the average accuracy of the classification tasks computed on the testing set of the UCI-HAR dataset. As expected, models with reduced complexity correspond to reduced computational loads which also lead to lower classification accuracies while the execution time is also strongly related to the computational performance of the device.

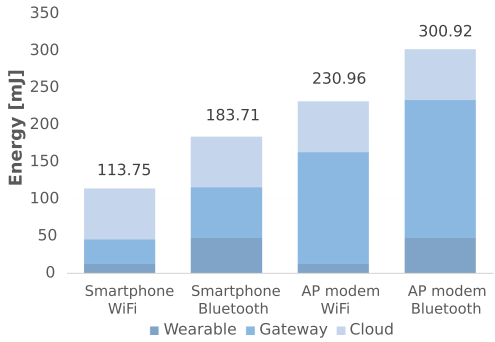

Fig. 5. Energy consumption of the traditional on-cloud inference paradigm.

VI. Experimental Results

After having characterized the energy model of the system under study in terms of:

1) power state diagram of the smartwatch-smartphone/AP-cloud distributed system;
2) compute performance (number of floating point operations per second) and power consumption of a set of possible devices;
3) throughput, power (Watt), and *latency* (msec) of different types of communication interfaces and links;
4) computational load, execution time, and accuracy of tasks executed by the system under different working hypotheses.

We can use the parameters obtained so far to plug in the energy simulator that we implemented for evaluating system performances.

Indeed, in the following of this section, we describe the proposed energy simulator and the experimental setup adopted for evaluation, and we finally report the results obtained from several numerical experiments.

A. Energy Simulator

Our Energy simulator arises from the functional state diagram shown in Fig. 2, which captures the behavior of the reference system for sensor-based HAR. The total energy consumed by the system is evaluated starting from (8) described in Section IV. Rather than evaluating the energy consumption as a function of the probability of exit at point eth (α_e), as described so far, we compute it in a real scenario starting from the classification models trained on top of the UCI-HAR dataset used to classify real samples for given values of the user-defined thresholds β_e. In particular, each sample belonging to the testing set has been processed through the EEoI-HAR network while the corresponding consumed energy has been calculated. Notice that, in order to reduce the degrees of freedom, in our simulations we applied the same threshold at each exit point (i.e., we assumed that $\beta_e = B,\ \forall e \in [1, 2, \ldots, K]$). In these conditions, we define the energy computation procedure reported in Algorithm 1 through which it is possible to calculate the Effective Inference Energy (EIE).

By iteratively executing this procedure on the samples belonging to the testing set we compute the average effective inference energy ($\overline{EIE}$) for a given B which can be used as a global metric to estimate the average energy consumption of the system. At the same time, also the *Effective Classification* ($EC(W_t)$) of a sample W_t for the chosen exit point is returned. Therefore, an EEoI network is more efficient the more often it can stop the inference phase early by testing a few exit points before finding the one that satisfies the confidence threshold. For instance, the best case occurs when the first exit point is already able to satisfy the confidence threshold. In this case, the $EIE(W_t)$ of the network will be equal only to the energy needed to sample data (E_{samp}) and to make the inference on the wearable device (E_{inf1}). On the other hand, in the worst case, no available exit points satisfy the confidence threshold so the effective classification is done by the backbone network (the last exit point). In this condition, $EIE(W_t)$ is the sum of the energy contribution of each device holding each exit point.

Algorithm 1: Effective Inference Energy (EIE) Computation.

K : number of exit points in the network
W_t : current sample
B : entropy threshold
$c_e(W_t)$: classification of sample w_t by the eth exit point
$EIE(W_t)$: effective inference energy of sample w_t

$e \leftarrow 2$
$EIE(W_t) \leftarrow E_{samp} + E_{inf_1}$
$EC(W_t) \leftarrow c_1(W_t)$
while $e <= K$ **do**
  if $H[c_e(W_t)] \geq B$ **then**
    break
  else
    $EIE(W_t) \leftarrow$
   $EIE(W_t) + E_{tx_{e-1}} + E_{rx_e} + E_{dec_e} + E_{inf_e}$
    $EC(W_t) \leftarrow c_e(W_t)$
    if $e == 2$ **then** $EIE(W_t) \leftarrow EIE(W_t) + E_{enc}$
    end if
  end if
  $e \leftarrow e + 1$
end while
return $EIE(W_t), EC(W_t)$

B. Experimental Setup

The energy simulator and the LSTM network have been implemented and tested on Matlab2022a platform running on a Windows desktop pc equipped with an Intel Core i9 and 16 GB of RAM. The network of Fig. 3 has been trained and tested using the UCI-HAR dataset [45] which contains activity data gathered from 30 participants of varying heights, weights, and ages (between 18 and 48 years). The dataset has been randomly partitioned into two sets: 70% was selected for generating the training data and 30% the test data, for a total of 7352 and 2947 number of records per dataset, respectively. It contains both accelerometer and gyroscope signals sampled at 50 Hz. The

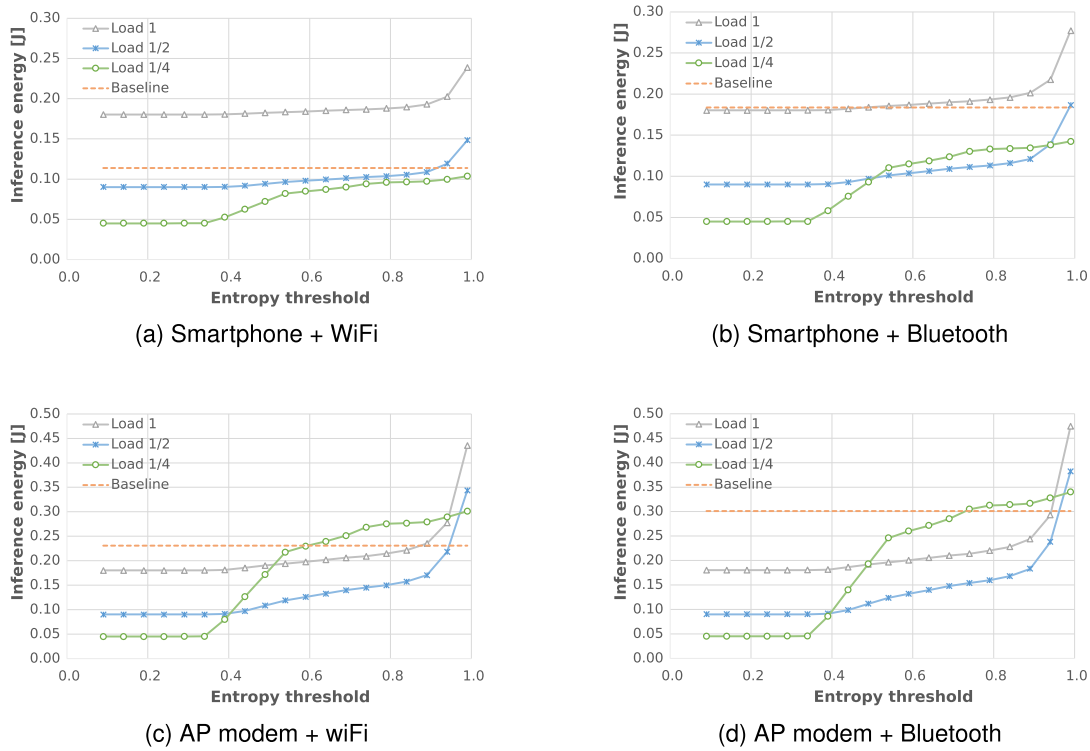

Fig. 6. EEoI-HAR system energy consumption for a single inference when varying the entropy threshold B.

signals have been divided into windows (W_t) containing 128 samples; given the sampling frequency of 50 Hz, this results in a 2.56 seconds time window. The monitored activities are: *walking, walking upstairs, walking downstairs, sitting, standing, and lying down*.

Notice that, we chose to completely decouple the training of the backbone network from those of the early exits sub-networks as presented by Laskaridis et al. in [5]. Once the models were trained, the classification performances of the complete network and of the EEoI approach were assessed using the testing samples. To increase the robustness of the results, the entire procedure has been repeated five times, randomly choosing which samples to insert in the training set and which ones in the test set.

C. Average Inference Energy

In the first set of experiments, we calculate the average energy needed to make an inference on a *baseline* system. Concerning the reference system described so far, the smartwatch can use both Bluetooth or WiFi to communicate with both the smartphone and the access point leading to four possible configurations.

Fig. 5 shows the energy consumed in the four configurations together with the amount attributable to each device. The most convenient strategy is to use the WiFi interface associated with using the smartphone as a gateway. This is due to the high energy efficiency of both WiFi and 4 G/5 G wireless technologies which over the years have been perfected to optimize communication in battery-powered devices. On the contrary, AP modems built on top of singleboard computers appear as highly energy-intensive devices with over 4 W of power consumption. Concerning Bluetooth technology, it must be noted that, although the power involved is very low, the low throughput that characterizes it imposes a higher transmission time which increases energy consumption. Another noteworthy piece of information that emerges from the figure concerns the fact that the energy consumed by the Cloud, to carry out the whole classification, is comparable, if not even lower, with that consumed by the devices that act as gateways. This is thanks to the enormous energy efficiency achieved by current GPUs dedicated to machine learning.

In the second set of experiments, we compared the $\overline{EIE}$ of the EEoI-HAR system, obtained when varying B, with the on-Cloud only inference values (*baseline*). Fig. 6 shows the comparison for the four system configurations previously described. Interestingly, in the "Smartphone + WiFi" configuration, only for the lowest load in charge of the smartwatch (i.e., *Load 1/4*), the EEoI technique turns out to be energetically convenient with respect to the baseline. Even with this reduced load, however, the gain is minimal and almost close to zero whenever a high classification accuracy requirement must be met ($B > 0.8$). Notice that, using gradually increasing accuracy thresholds forces the EEoI system to test all available exit points often without being able to meet the accuracy requirement. Obviously, this involves an increase in global energy consumed, due to the useless inference energy contributions, compared to the baseline. Here, in fact, all the intermediate exit points only act as gateways by forwarding the data without investing energy in the inference phase.

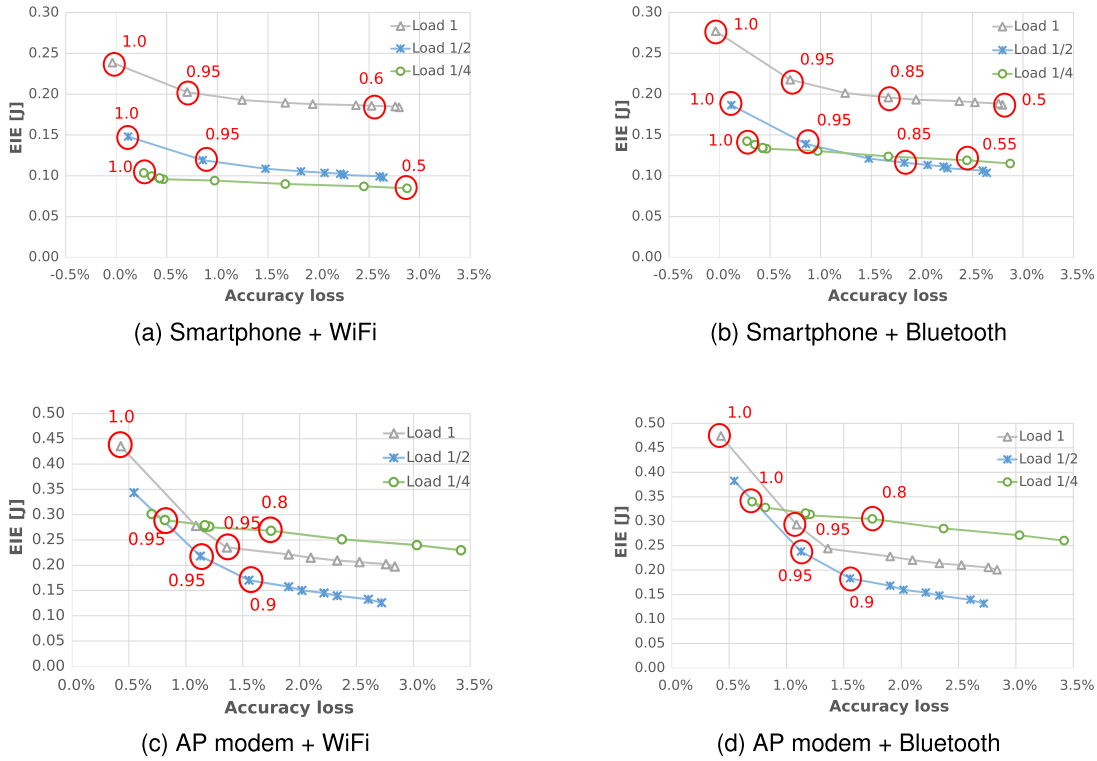

Fig. 7. Pareto curves reporting accuracy loss versus average inference energy when varying the value of the entropy threshold B.

In the "Smartphone + Bluetooth" case, the energy saving appears more evident both for *Load 1/2* and *Load 1/4* while, with a full load, the baseline still turns out to be the most convenient.

Definitely, in cases where the smartphone acts as a gateway the choice to overcharge the wearable device with a heavy task is not recommended due to the fact that the gateway shows a strong energy efficiency both in terms of communication and model execution so it is better to exploit it more.

On the other hand, when using the AP modem as the gateway, the EEoI technique results in a more evident energy saving due to the increased cost of the transmission. Moreover, it is interesting to see that the lowest load (*Load 1/4*) in charge of the smartwatch is not, this time, the energetically best strategy because the singleboard computer which builds an AP modem does not offer particular computing power and energy efficiency, thus delegating a greater classification activity to this is not convenient.

D. Energy Accuracy Tradeoff

Starting from the $EC(W_t)$ of the testing samples, we computed the classification accuracy of the system for a given value of B using

$$Accuracy = \frac{1}{N}\sum_{i=1}^{N}\frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i}, \qquad (10)$$

where N is the number of classes to be recognized, $i \in [1 \cdots N]$ is an index that identifies a specific class from which it is possible

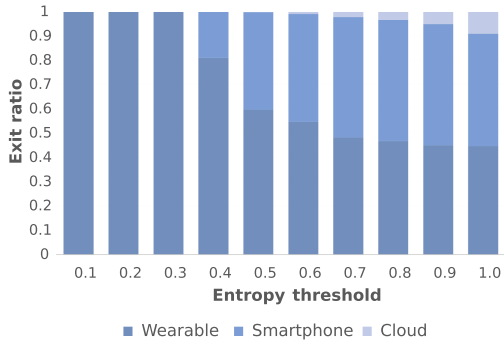

Fig. 8. Exit ratio for each point using the smartphone as a gateway when varying the entropy threshold B.

to evaluate the following quantities: TP_i, the number of true positives predicted for class i; TN_i, the number of true negatives predicted for class i; FP_i, the number of false positives predicted for class i; FN_i, the number of false negatives predicted for class i.

Fig. 7 shows, for each configuration of the system, the Pareto curves plotting the classification loss, with respect to the baseline, versus the $\overline{EIE}$ obtained when varying B. To increase readability, some representative values of the entropy threshold B have been highlighted by means of red circles. Notice that, when B is equal to zero, the network always stops the inference at the first exit so its classification accuracy is equal

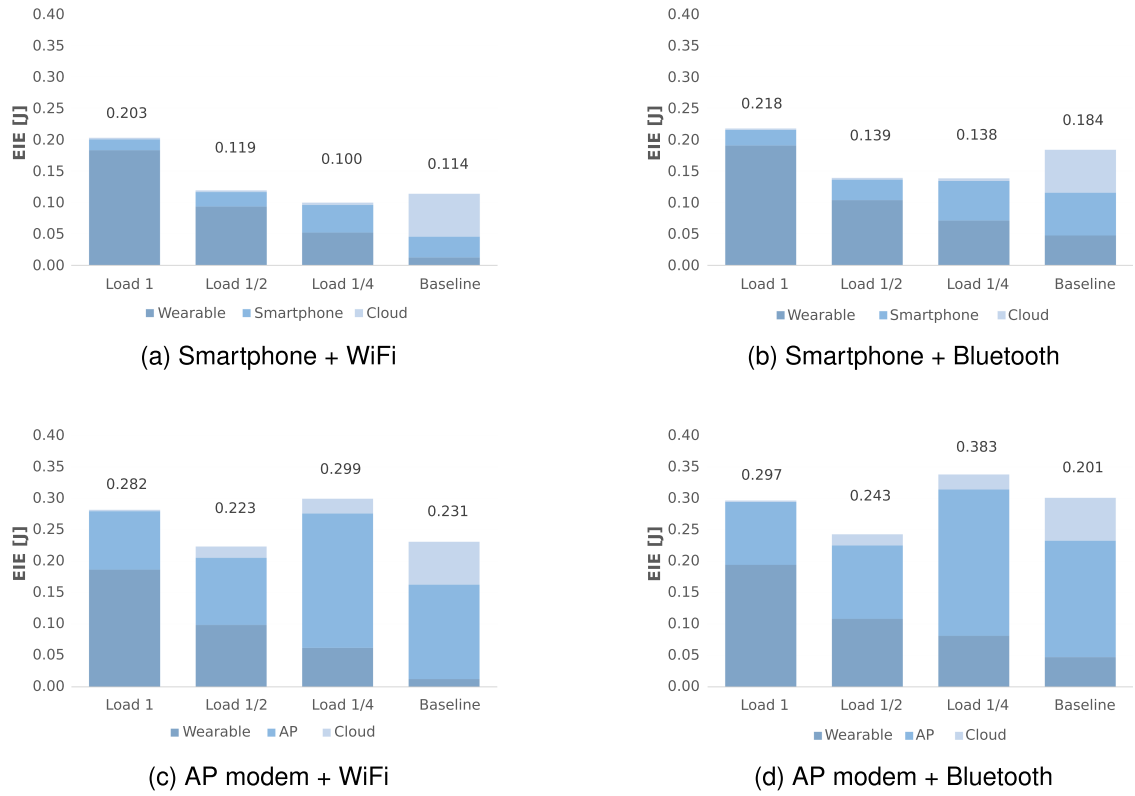

Fig. 9. Average energy distribution for an entropy threshold B of 0.95.

to that of the first network branch. For instance, Fig. 8 shows the exit ratio for the three exit points measured when varying the entropy threshold B for the system configuration in which the smartphone acts as a gateway. Until the value of B exceeds 0.3, the only exit point used is the first one (i.e., the inference is made locally to the wearable device). Moreover, further increasing the value of B entails an ever greater involvement of subsequent exit points. On the other hand, setting the threshold to 1 does not force always exiting at the backbone network. In fact, even with the threshold set at the maximum value, the network can still stop at early exits if the prediction confidence for that sample reaches the maximum value.

Obviously, the greater the B, the greater the likelihood of using the furthest exits, and potentially, the effectiveness of the network. On the other hand, using more and more far exits, while increasing the effectiveness in terms of accuracy, decreases the efficiency in terms of consumed energy. Finally, it is interesting to note that, in some cases, the average accuracy obtained with the EEoI technique exceeds the baseline value (negative loss value) as for some samples the early branches were able to classify better than the backbone. Notice that this derives from the training strategy we have used which considers each submodel as a separate network. Therefore, on some input samples it may happen that an early exit point has better performance than the backbone and, since using high values of B actually means for each sample looking for one of the best exit points, it may happen that the EEoI strategy can reach better classification performances with respect to the backbone. For sake of readability, we reported the results obtained with the values of B from 0.5 to 1.

With the exception of the curve obtained with the minimum load which is almost flat, it is interesting to note that the optimal point does not correspond to the maximum value of B. In fact, a value of B ranging from 0.9 or 0.95 often corresponds to the best tradeoff between loss of accuracy and energy cost.

To deeply investigate the system behavior, we report, in Fig. 9, the average energy distribution for an entropy threshold B of 0.95 compared with the baseline. As expected, the wearable task's computational load directly influences the energy consumed by the smartwatch and indirectly that consumed by the gateway and by the Cloud. From the wearable device point of view, the strategy of delegating complex tasks to the gateway or the Cloud appears to be the winning choice. But, from the point of view of the energy consumed globally, the optimal solution would be to accurately balance the load at the various nodes taking into account their energy efficiency and the cost needed to reach them.

Concerning the configurations in which the smartphone acts as a gateway, for example, the high classification accuracy of the model executing on it limits to few samples the need to forward the data to the Cloud. In this condition, reducing the load of the smartwatch task can reduce the global energy consumed by the system making the EEoI technique energetically advantageous compared to the baseline. Furthermore, even playing with the extent of the smartwatch load can be useful to drive the energy

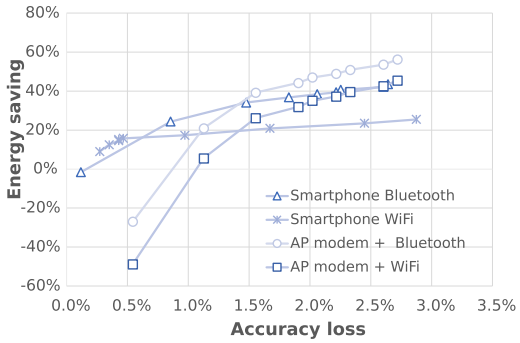

Fig. 10. Energy saved versus accuracy loss when varying the entropy threshold B.

balance between smartphone and smartwatch without impacting the total energy. In fact, being both personal and portable devices, it could be useful, in some circumstances, to increase the workload of the smartphone to save the smartwatch battery and vice versa.

In the case of the infrastructured systems which used traditional AP modems, reducing the smartwatch workload appreciably increases the energy contribution requested to the Cloud due to the fact that the classifier installed on the access point can not always compensate for the diminished classification capacity of the wearable device. Even in this case, despite everything, with an appropriate dimensioning of the workload, it is still possible to reduce the total energy consumption (*Load 1/2*).

Finally, in Fig. 10 the energy saving of the four system configurations, obtained with different values of B, is plotted versus the accuracy loss with respect to the corresponding baselines. Although it is evident that the EEoI technique does not guarantee an energy gain for accuracy loss values close to zero, admitting a loss of even just 1.5% it is possible to save up to 40% of the total energy.

VII. Conclusion

In this work, we presented an investigation pertaining to the energy budget required to sustain deep learning-based inference applied to human activity recognition. In particular, we studied and compared different configurations from an early-exit perspective, where the architecture of these deep learning models is implemented in a distributed manner. Exit points have been hypothesized to be placed: i) on board of wearable devices; ii) on typical edge hardware acting as gateways (i.e., smartphone and access points); iii) on the cloud back-end. The effect of different types of communication channels has also been explored, by taking into consideration the type of interface, namely: Bluetooth, WiFi, 4G/5G for wireless communication, and DSL or optical fiber for a cable-based.

In order to evaluate the overall energy consumption and its breakdown along the different devices involved, we derived an analytical model of energy expenditure that we used jointly with power consumption figures of common hardware platforms derived from datasheets. Then, we built a simulator to evaluate the energy required to perform a given HAR task by means of early-exit neural networks under different configurations and compared them with a reference baseline with the transmission of data samples to the cloud for subsequent classification.

The experimental results provide evidence of a subtle interplay between the costs due to communication and those to be paid for computation, which clearly depends on the underlying physical communication link, the type of hardware devices adopted as early exit points of the neural network, and the samples on which inference is performed.

For instance, we found that deep learning models implemented in a distributed way with early exits do not always ensure energy reductions with respect to a cloud-based solution if the same accuracy levels are kept; nonetheless, by tolerating just a 1.5% decrease in accuracy, it is possible to achieve a reduction around 40% of the total energy consumption.

References

[1] B. Soret et al., "Learning, computing, and trustworthiness in intelligent IoT environments: Performance-energy tradeoffs," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 629–644, Mar. 2022.
[2] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.
[3] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, and A. Y. Zomaya, "Edge intelligence: The confluence of edge computing and artificial intelligence," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7457–7469, Aug. 2020.
[4] A. E. Varjovi and S. Babaie, "Green Internet of Things (GIoT): Vision, applications and research challenges," *Sustain. Comput. Informat. Syst.*, vol. 28, 2020, Art. no. 100448.
[5] S. Laskaridis, A. Kouris, and N. D. Lane, "Adaptive inference through early-exit networks: Design, challenges and directions," in *Proc. 5th Int. Workshop Embedded Mobile Deep Learn.*, 2021, pp. 1–6.
[6] C. Jiang et al., "Energy aware edge computing: A survey," *Comput. Commun.*, vol. 151, pp. 556–580, 2020.
[7] F. Samie, L. Bauer, and J. Henkel, "Hierarchical classification for constrained IoT devices: A case study on human activity recognition," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8287–8295, Sep. 2020.
[8] F. Daghero, D. J. Pagliari, and M. Poncino, "Two-stage human activity recognition on microcontrollers with decision trees and CNNs," in *Proc. 17th Conf. PhD Res. Microelectronics Electron.*, 2022, pp. 173–176.
[9] M. Odema, N. Rashid, and M. A. Al Faruque, "EExNAS: Early-exit neural architecture search solutions for low-power wearable devices," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Des.*, 2021, pp. 1–6.
[10] G. Bhat, N. Tran, H. Shill, and U. Y. Ogras, "w-HAR: An activity recognition dataset and framework using low-power wearable devices," *Sensors*, vol. 20, no. 18, pp. 1–26, 2020.
[11] N. Rashid, B. U. Demirel, and M. A. Al Faruque, "AHAR: Adaptive CNN for energy-efficient human activity recognition in low-power edge devices," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13041–13051, Aug. 2022.
[12] D. N. Jha et al., "A hybrid accuracy-and energy-aware human activity recognition model in IoT environment," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 1, pp. 1–14, First Quarter 2023.
[13] Z. Hussain, D. Waterworth, M. Aldeer, W. E. Zhang, and Q. Z. Sheng, "Toothbrushing data and analysis of its potential use in human activity recognition applications: Dataset," in *Proc. 3rd Workshop Data: Acquisition Anal.*, 2020, pp. 31–34.
[14] E. Lattanzi, L. Calisti, and V. Freschi, "Unstructured handwashing recognition using smartwatch to reduce contact transmission of pathogens," *IEEE Access*, vol. 10, pp. 83111–83124, 2022.
[15] A. Jordao, L. A. B. Torres, and W. R. Schwartz, "Novel approaches to human activity recognition based on accelerometer data," *Signal, Image Video Process.*, vol. 12, no. 7, pp. 1387–1394, 2018.
[16] Y. Wang, S. Cang, and H. Yu, "A survey on wearable sensor modality centred human activity recognition in health care," *Expert Syst. Appl.*, vol. 137, pp. 167–190, 2019.

[17] X. Zhou et al., "Deep-learning-enhanced human activity recognition for Internet of Healthcare Things," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6429–6438, Jul. 2020.
[18] R. Saini, P. Kumar, P. P. Roy, and D. P. Dogra, "A novel framework of continuous human-activity recognition using kinect," *Neurocomputing*, vol. 311, pp. 99–111, 2018.
[19] J. Manjarres, P. Narvaez, K. Gasser, W. Percybrooks, and M. Pardo, "Physical workload tracking using human activity recognition with wearable devices," *Sensors*, vol. 20, no. 1, pp. 1–17, 2019.
[20] S. C. Akkaladevi and C. Heindl, "Action recognition for human robot interaction in industrial applications," in *Proc. IEEE Int. Conf. Comput. Graph. Vis. Inf. Secur.*, 2015, pp. 94–99.
[21] D. R. Beddiar, B. Nini, M. Sabokrou, and A. Hadid, "Vision-based human activity recognition: A survey," *Multimedia Tools Appl.*, vol. 79, no. 41, pp. 30509–30555, 2020.
[22] O. D. Lara and M. A. Labrador, "A survey on human activity recognition using wearable sensors," *IEEE Commun. Surveys Tut.*, vol. 15, no. 3, pp. 1192–1209, Third Quarter 2013.
[23] N. Kale, J. Lee, R. Lotfian, and R. Jafari, "Impact of sensor misplacement on dynamic time warping based human activity recognition using wearable computers," in *Proc. Conf. Wireless Health*, 2012, pp. 1–8.
[24] J. Kulchyk and A. Etemad, "Activity recognition with wearable accelerometers using deep convolutional neural network and the effect of sensor placement," in *Proc. IEEE SENSORS*, 2019, pp. 1–4.
[25] I. Y. Jung, "A review of privacy-preserving human and human activity recognition," *Int. J. Smart Sens. Intell. Syst.*, vol. 13, no. 1, pp. 1–13, 2020.
[26] E. Davarci, B. Soysal, I. Erguler, S. O. Aydin, O. Dincer, and E. Anarim, "Age group detection using smartphone motion sensors," in *Proc. 25th Eur. Signal Process. Conf.*, 2017, pp. 2201–2205.
[27] N. O'Mahony et al., "Deep learning vs. traditional computer vision," in *Proc. Sci. Inf. Conf.*, Springer, 2019, pp. 128–144.
[28] Y. Lai, "A comparison of traditional machine learning and deep learning in image recognition," *J. Phys. Conf. Ser.*, vol. 1314, no. 1, 2019, Art. no. 012148.
[29] N. G. Paterakis, E. Mocanu, M. Gibescu, B. Stappers, and W. van Alst, "Deep learning versus traditional machine learning methods for aggregated energy demand prediction," in *Proc. IEEE PES Innov. Smart Grid Technol. Conf. Europe*, 2017, pp. 1–6.
[30] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.
[31] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: Alexnet-level accuracy with 50x fewer parameters and $<$ 0.5 MB model size," 2016, *arXiv:1602.07360*.
[32] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1251–1258.
[33] M. Zhu and S. Gupta, "To prune, or not to prune: Exploring the efficacy of pruning for model compression," 2017, *arXiv:1710.01878*.
[34] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," *Neurocomputing*, vol. 461, pp. 370–403, 2021.
[35] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021.
[36] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. 23rd Int. Conf. Pattern Recognit.*, 2016, pp. 2464–2469.
[37] X. Wang, Y. Luo, D. Crankshaw, A. Tumanov, F. Yu, and J. E. Gonzalez, "IDK cascades: Fast deep learning by learning not to overthink," 2017, *arXiv:1706.00885*.
[38] Primate Lab, "Geekbench 4," 2022. Accessed: Nov. 14, 2022. [Online]. Available: https://www.geekbench.com/geekbench4/
[39] P. Ruiu, C. Fiandrino, P. Giaccone, A. Bianco, D. Kliazovich, and P. Bouvry, "On the energy-proportionality of data center networks," *IEEE Trans. Sustain. Comput.*, vol. 2, no. 2, pp. 197–210, Second Quarter 2017.
[40] D. Nörtershäuser et al., "An empirical study of power characterization approaches for servers," in *Proc. 9th Int. Conf. Smart Grids Green Commun. IT Energy-Aware Technol.*, 2019, pp. 1–6.
[41] J. Baliga, R. Ayre, K. Hinton, and R. S. Tucker, "Energy consumption in wired and wireless access networks," *IEEE Commun. Mag.*, vol. 49, no. 6, pp. 70–77, Jun. 2011.
[42] B. Dusza, C. Ide, L. Cheng, and C. Wietfeld, "CoPoMo: A context-aware power consumption model for lte user equipment," *Trans. Emerg. Telecommun. Technol.*, vol. 24, no. 6, pp. 615–632, 2013.
[43] Y. Sun, N. B. Agostini, S. Dong, and D. Kaeli, "Summarizing CPU and GPU design trends with product data," 2019, *arXiv:1911.11313*.
[44] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Compute and energy consumption trends in deep learning inference," 2021, *arXiv:2109.05472*.
[45] J.-L. Reyes-Ortiz, L. Oneto, A. Samà, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, 2016.

Emanuele Lattanzi received the laurea degree, in 2001 and the PhD degree from the University of Urbino, Italy, in 2005. Since 2001, he has been with the Information Science and Technology Institute, University of Urbino. In 2003, he was with the Department of Computer Science and Engineering, Pennsylvania State University, as a visiting scholar with Prof. V. Narayanan. From 2008 until 2020, he has been Assistant Professor in computer engineering with the Department of Pure and Applied Sciences (DiSPeA), University of Urbino, where he is currently an associate professor in computer engineering. His research interests include wireless sensor networks, energy-aware routing algorithms, machine learning, and Internet of Things.

Chiara Contoli received the graduated degree in computer engineering from the University of Bologna, Italy, in 2013, and the PhD degree from the University of Bologna in Electronics, Telecommunications and Information Technologies Engineering, in 2017. For two years, she worked as a software developer in the industry. Since 2022, she is a fixed-term junior Assistant Professor in computer engineering with the Department of Pure and Applied Sciences (DiSPeA), University of Urbino, Italy. Her research interests are in network management and network softwarization, network architectures and protocols, the Internet of Things, machine learning, and power management. She is also interested in programming languages for networks and enjoys working on interdisciplinary problems.

Valerio Freschi received the laurea degree in electronic engineering from the University of Ancona, Italy, in 1999, and the PhD degree in computer science engineering from the University of Ferrara, Italy, in 2006. He is currently an Associate Professor in computer engineering with the Department of Pure and Applied Sciences (DiSPeA), University of Urbino, Italy. His research interests include wireless sensor networks, energy-efficient algorithms, optimization, and machine learning.